Codex CLI 讲义摘要

适合培训、复习、分享的速读版

第一次上手

权限与审批

提示词速记

这份短版保留了最核心的起步路径：安装、提问、权限判断、常用命令、稳定工作流、排错顺序和最后的速记卡。

请以你本地 `codex --help` 和 OpenAI 官方 Codex 文档为准。

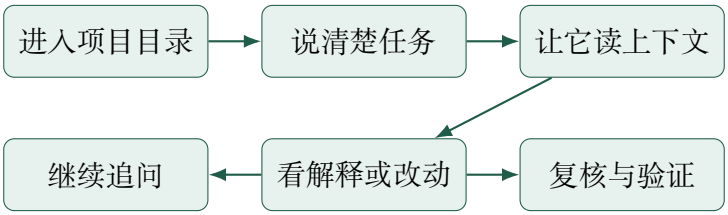
1

一页理解

先记住一句最实用的话：**Codex CLI 是一个在终端里工作的 AI 编码代理**。

它最适合做四类事：

- 看懂陌生项目；
- 定位功能和报错；
- 做小范围改动；
- 做代码审查、总结、给出验证建议。



速记

第一次用 Codex CLI，不求全会。先掌握一条线：进目录、提具体任务、继续追问、最后验证。

2

第一次上手

2.1 安装与登录

```
npm i -g @openai/codex
codex --help
codex
```

如果你想显式登录或检查状态：

```
codex login
codex login status
```

如果你走 API 密钥路线：

```
printenv OPENAI_API_KEY | codex login --with-api-key
```

记住这一点

这两个变量名别混：上面这条命令是在“登录”时从 `OPENAI_API_KEY` 读入密钥；到了 `codex exec` 和 CI 场景，官方更常用的是 `CODEX_API_KEY`。

2.2 Linux 环境下先确认这四件事

下面默认你用的是 Linux 终端。无论你是在本地、SSH 到远端，还是在容器里，第一次开始前都建议先跑：

```
pwd
ls
codex --help
codex login status
```

这四步分别对应四个关键问题：我现在在哪、这是不是目标项目、CLI 能不能用、认证是不是已经就绪。

2.3 macOS、Windows、WSL 一句话提醒

- **macOS**：整体和 Linux 很像，先确认 `codex` 在 PATH 里。
- **PowerShell**：环境变量写法和 Bash 不一样，别直接照抄。
- **WSL**：把它当独立 Linux；如果项目主要在 WSL 里，就尽量在 WSL 里完成登录、Git 和测试。

别踩这个坑

最容易乱的是 Windows 和 WSL 混着用。项目主要在哪边，就尽量整轮任务都在哪边完成。

2.4 第一次提问

```
请先看一下当前项目结构，告诉我入口文件、核心目录和推荐阅读顺序，并请引用具体文件路径。
```

别踩这个坑

不要一上来就说“帮我改好”或“帮我优化一下”。范围太宽，结果通常会变空。

3

最常用命令

把第一句任务说清楚以后，下一步就是认入口：什么时候直接进会话，什么时候跑单次命令，什么时候留在会话里用斜杠命令。先别把这一页当命令大全来背，先把“眼前这个场景该走哪条路”搞明白。

命令	作用	什么时候用
codex	交互式会话	本地日常开发最常用
codex exec "..."	非交互执行	脚本、CI（持续集成）、一次性任务
codex review --uncommitted	审查当前改动	提交前快速找风险
codex resume --last	接着上次会话干	不想重复讲背景时
codex apply <TASK_ID>	应用某次任务 diff	结果要落到工作树时
/new	开一轮新会话	不退出终端，直接重开一轮
/clear	清屏并新开会话	想清掉当前屏幕内容时
/resume	恢复旧会话	想切回之前保存的会话
/fork	从旧会话分叉	想保留上下文，但开一条新线
/status	查看会话状态	看模型、权限、可写目录
/permissions	调整权限模式	切换更保守或更自动
/review	会话内做代码审查	先查风险再看 diff
/diff	查看改动	改完先审一眼
/plan	先出计划	大任务先计划再动手
/agent	查看或切换 agent 线程	用了 subagent 后看并行线程和细节
/init	生成 AGENTS.md 脚手架	想沉淀仓库规则时

记住这一点

如果你只背三个入口，先背 codex、codex exec、codex review。

3.1 现在该用哪个入口？

- 想连续对话、边看边追问：codex
- 想脚本化、跑 CI、做一次性任务：codex exec
- 已经在会话里，想原地重开：/new 或 /clear
- 想回到旧会话：/resume 或 codex resume --last
- 想保留旧思路，再开一条新线：/fork

3.2 什么时候开 `--search`

- 只看本地代码：项目结构、调用链、当前分支改动、仓库里的测试命令。
- 开 `--search`：官方文档、依赖版本差异、外部 API、最新错误说明。

记住这一点

先问本地事实，再补外部资料。并要求把“仓库事实”和“外部来源”分开写。

4

权限与审批

4.1 三个沙箱级别

- `read-only`：只读，不改。
- `workspace-write`：能改当前工作区，最适合本地开发。
- `danger-full-access`：权限很大，只在你非常清楚风险时使用。

4.2 三个审批策略

- `untrusted`：安全读操作自动跑，不信任命令先问你。
- `on-request`：由代理判断什么时候该申请批准。
- `never`：不弹审批，常用于自动化，但前提是你自己已经控好风险。

```
codex --full-auto
```

它相当于一个比较省事的默认组合：

```
--sandbox workspace-write --ask-for-approval on-request
```

速记

本地开发优先从 `workspace-write` 起步。先别急着全放开。

5

提示词速记

5.1 一条好提示词的四要素

1. 目标：你到底要它做什么。
2. 范围：只围绕哪些文件、功能或问题。
3. 限制：不要做什么。
4. 输出格式：你希望它最后怎么汇报。

5.2 直接可用模板

看项目：

```
请按新手视角介绍当前项目，重点说入口、核心目录、主流程和推荐阅读顺序。
```

定位功能：

```
请帮我定位登录相关代码，告诉我入口、涉及文件和大致调用链。
```

处理报错：

```
这是报错信息。请先解释最可能原因，再给排查顺序，最后给最小修改方案。
```

小改动：

```
请只修改提示文案，不要改业务逻辑。
修改后列出改动文件和验证方法。
```

6

稳定工作流

这一节是把前面那些零散动作串成一条主线：先理解、再定位、再小改、最后复核。后面的“会话控制”和“自动化与 CI”更像按需补充，不必第一次就全背下来。

6.1 本地开发推荐顺序

1. 先解释项目或问题；
2. 再定位文件和调用链；
3. 再做小范围修改；
4. 改完跑 `/review`；
5. 再看 `/diff`；
6. 最后自己验证。

6.2 改坏了先怎么救

```
git status
git diff
git switch -c backup/codex-check
git stash push -u -m "before-rollback"
```

先看现场，再保留现场，最后才决定撤哪一部分。能只撤一个文件，就别一上来整批清空。

6.3 三轮追问示例

1. 第一轮拿地图：先问入口文件、相关目录、阅读顺序。
2. 第二轮抓主线：只盯一个功能，让它列关键文件和调用链。
3. 第三轮要交付：只改一个小点，并要求按“改动文件 / 验证 / 风险”总结。

6.4 任务完成验收卡

一轮任务结束前，至少自己确认四件事：

- 改动范围有没有控住；
- 实际跑了哪些验证；
- 还有什么剩余风险；
- 现在适不适合提交。

如果你想让 Codex 按这个格式汇报，可以直接说：

```
请按“改动文件 / 验证情况 / 剩余风险 / 是否建议现在提交”四项总结。
```

6.5 为什么要用 AGENTS.md

AGENTS.md 适合沉淀这些长期规则：

- 仓库结构和重点目录；
- 构建、测试、静态检查命令；
- 哪些文件不要动；
- 交付时怎么总结。

```
/init
```

记住这一点

如果你在三次任务里都重复说过同一句话，那它大概率就该进 AGENTS.md 了。

6.6 个人默认习惯写进 config.toml

AGENTS.md 管仓库规则；config.toml 管你自己的默认习惯，比如常用模型、审批和沙箱组合。

最常用的位置：

- 用户级：~/.codex/config.toml
- 项目级：.codex/config.toml

最值得先认的几类信息：

- 顶层：model、approval_policy、sandbox_mode、web_search
- [profiles.<name>]：常用模式
- [features]：功能开关
- [agents]：subagent 并发和层级

```
codex -p safe
codex -p dev
codex --oss --local-provider ollama -p local
```

先想清楚你最常用的 2 到 3 种模式，再去固化。

```
codex features list | rg multi_agent
codex features enable multi_agent
```

记住这一点

当前版本里 multi_agent 通常默认就是开的。多数时候不是“没开”，而是你没有明确要求它去 spawn subagent。

7

会话控制

7.1 上下文太长怎么办？

优先做四件事：

1. 用 /compact 压缩长会话；
2. 用 /mention 点名关键文件；
3. 把任务缩到一个功能或一个报错；
4. 真想重开一轮，就先用 /new；想清屏就用 /clear。

7.2 继续旧会话和新开会话怎么选？

- 想原地开启新会话：/new
- 想清屏后再开：/clear
- 想切回旧会话：/resume
- 想基于旧会话分一条新线：/fork
- 想从命令行里直接接着最近一次聊：codex resume --last

7.3 能在多个会话间切换吗？

可以，但更像“恢复某个已保存会话”，不是像浏览器标签页那样一直挂着来回切换。日常最常用的是 /resume、/fork，以及命令行里的 codex resume。

7.4 对照图

/new

原地开新会话。
适合：不退出终端，直接重开。

/clear

清屏后再开新会话。
适合：屏幕太乱，想顺手清掉。

/resume

恢复旧会话。
适合：切回上次的上下文。

/fork

从旧会话分叉。
适合：保留旧讨论，再开新线。

8

自动化与 CI

前面讲的，都是你人在终端前盯着会话用的情况。下面切到另一类场景：不连续对话，而是把一次性任务塞进脚本或 CI。

8.1 最小示例

```
export CODEX_API_KEY=your_api_key

codex exec --json \
  -o codex-summary.txt \
  "审查当前仓库，并总结最主要的 3 个风险点"
```

8.2 这三个参数分别干什么

- `CODEX_API_KEY`：给非交互执行做认证。
- `--json`：让输出更适合脚本读取。
- `-o`：把最后一条摘要单独写到文件里。

8.3 脚本闭环怎么搭

```
codex exec --json \
  --output-schema review-schema.json \
  -o codex-summary.txt \
  "审查当前改动" > codex-events.jsonl
```

给人看：`codex-summary.txt`。给脚本读：`codex-events.jsonl`。要稳定结构：`--output-schema`。

别踩这个坑

自动化更适合做总结、分类、只读检查。大范围自动改代码，先在本地跑稳再说。

9

排错与避坑

9.1 真实排错顺序

当你遇到报错时，最稳的顺序通常是：

1. 先让它解释错误类型；
2. 再让它定位最可能的文件；
3. 再让它给最小修复方案；
4. 最后让它说明验证方法。

9.2 最容易踩的坑

- 没确认当前目录就开始操作。
- 任务太宽，结果很空。
- 一上来就给太大权限。
- 改完不看 `/diff`，也不做验证。
- 在 CI 里直接照搬交互式习惯。

9.3 高风险边界

下面这些事，别一上来就让 Codex 直接放手执行：

- 删除、覆盖、批量改名、批量格式化；
- 破坏性 Git 操作；
- 数据库迁移、数据回填、修复脚本；
- 密钥、认证缓存、生产配置；
- 会触发外部副作用的命令。

更稳的做法是先说：先不要执行，先列风险、影响范围、回滚方法。

速记

新手最大的问题不是不会敲命令，而是没有形成节奏：确认目录、收紧范围、审改动、做验证。

10

FAQ

10.1 上下文窗口超了怎么办？

不要继续硬塞更多内容，优先这样做：

1. 缩小任务范围，只盯一个功能或一个报错。
2. 用 `/compact` 压缩长会话。
3. 只点名真正关键的文件，不要一次拉太多目录。
4. 必要时开新会话，并先用几句话总结上一轮结论。

10.2 它回答得很空怎么办？

大多数时候是任务太宽。把问题改成“哪几个文件、哪个功能、最后要什么格式”，通常会立刻好很多。

10.3 最少要做哪些复核？

如果你只做最低限度检查，至少做三件事：看 `/diff`、跑相关测试或静态检查、自己验证关键路径。时间允许的话，再补一次 `/review` 会更稳。

11

高手常用招式

到这里，入门主线其实已经够你正常开工了。下面这部分更像“用熟以后再加上的控场技巧”，适合按需查，不用第一次就全部照做。

11.1 这 9 招最值得记

1. 先比方案，再决定改不改。
2. 大仓库先收文件范围，不要一口气全讲。
3. 用 `/fork` 保留旧结论，再试另一条线。
4. 用 `/model` 做“先快后深”的分工。
5. 先定义成功标准，再开始修改。
6. 把代码审查做成闭环，不是只跑一次。
7. 固定交付模板，别让结果每次都飘。
8. `codex exec` 优先做总结、分类、只读检查。
9. 任务能拆时，显式要求它用 subagent。

11.2 直接可抄的进阶提示词

方案评审：

```
先不要改代码。请给我 2 到 3 种实现方案，分别说明改动范围、优点、风险，并给推荐。
```

大仓库控上下文：

```
先不要全仓库泛讲。请先找出和“登录失败提示”最相关的 5 个文件，只输出路径，并告诉我下一轮先追哪两个。
```

成功标准前置：

```
先不要修改。请先复述目标、列出不做的事、给出完成标准和验证方法。等我确认后再开始。
```

结构化交付：

```
请固定按“结论 / 改动文件 / 验证情况 / 剩余风险 / 下一步建议”汇报。
```

11.3 最常见的高阶误用

- 明明该先比方案，却一上来就让它直接改。
- 明明仓库很大，却还在不断塞更多上下文。
- 跑了 `/review` 就想一次性改完所有建议。
- 想自动化大范围改动，却还没在本地手工跑稳。

11.4 MCP 先跑一个小例子

```
codex mcp list
codex mcp add docs --url https://example.com/mcp
```

最值得先试的，不是“全接上”，而是让 Codex 同时看本地代码和一个只读外部来源，比如产品文档或内部知识库。

记住这一点

第一次接 MCP，先做对照和解释，不要一上来就给外部写权限。

11.5 subagent 怎么上手

```
请 spawn 一个 subagent 负责一个检查点：
1. 安全
2. 真实 bug
3. 测试遗漏
等它们都完成后，再统一总结。
```

```
/agent
```

`/agent` 适合看当前有哪些 agent 线程，也方便切过去看细节。

11.6 怎么开多个 subagent

```
[agents]
max_threads = 6
max_depth = 1
```

`max_threads` 控同时并行多少个；`max_depth` 控能不能继续往下再 spawn。大多数时候先调线程数，不要急着加深层级。

11.7 自定义 subagent 放哪里

- 个人全局：`~/.codex/agents/`
- 当前项目：`.codex/agents/`

最小自定义文件至少写：`name`、`description`、`developer_instructions`。

12

练习题

12.1 练习一

让 Codex 给出一个公开小仓库的入口文件、核心目录和推荐阅读顺序。

12.2 练习二

选一个功能，例如登录或上传，让它解释入口、涉及文件和调用链。

12.3 练习三

只做一个低风险改动，例如改文案、补说明或补一条简单测试。

12.4 练习四

跑一遍：

```
/review
/diff
```

然后判断它给出的审查结果，是否真的落到了当前改动上。

12.5 练习五

把一句模糊请求改写成更清楚的版本。比如把“帮我优化一下这里”改成包含目标、范围、限制、输出格式的提示词，再比较前后效果。

12.6 做完后怎么判断合不合格？

如果你做题前就想先看判分标准，也完全可以先读这一节。

- 练习一至少要给出具体路径、推荐顺序、为什么先看这些文件。
- 练习二至少要讲清入口、关键文件和调用链，不只是列文件名。
- 练习三至少要说清改了哪些文件、没改什么、怎么验证。
- 练习四至少要把真正的问题和可选优化分开。
- 练习五至少要把目标、范围、限制、输出格式写清楚。

13

最后只记这几句

3 分钟背完

1. 日常开发多用 `codex`，脚本化多用 `codex exec`。
2. 第一次上手先做最小任务，不要急着大改。
3. 好提示词要写清楚目标、范围、限制、输出格式。
4. 本地开发先从 `workspace-write` 起步。
5. 改完一定先 `/review`，再 `/diff`，最后自己验证。
6. 高风险任务先看计划、风险和回滚，再决定要不要放行。
7. 有重复规则就写进 `AGENTS.md`。
8. 查本地问题先看仓库，真要最新资料再开 `--search`。
9. 想让它并行干活，得明确要求 spawn subagent，并先控住 `max_threads`。